

16-bit Arithmetic Logic Unit (ALU)

Student name	Student code
Aly Khaled Aly	91240475
Ahmed Amir elsayed hamdy	91240066

Table of Contents:

Abstract.....	3
Introduction.....	4
Design Approach.....	4
Verification Strategy.....	5
RTL code.....	6
Testbench code.....	9
Waveform.....	15
Transcript.....	15
Elaborating Design.....	16
Conclusion.....	17

Abstract:

This project presents the design and implementation of a 16-bit Arithmetic Logic Unit (ALU) in Verilog. The ALU supports arithmetic, logical, and shift operations through a modular architecture consisting of separate functional blocks and parameterized multiplexers. A complete testbench is used to verify correctness and generate status flags for each operation.

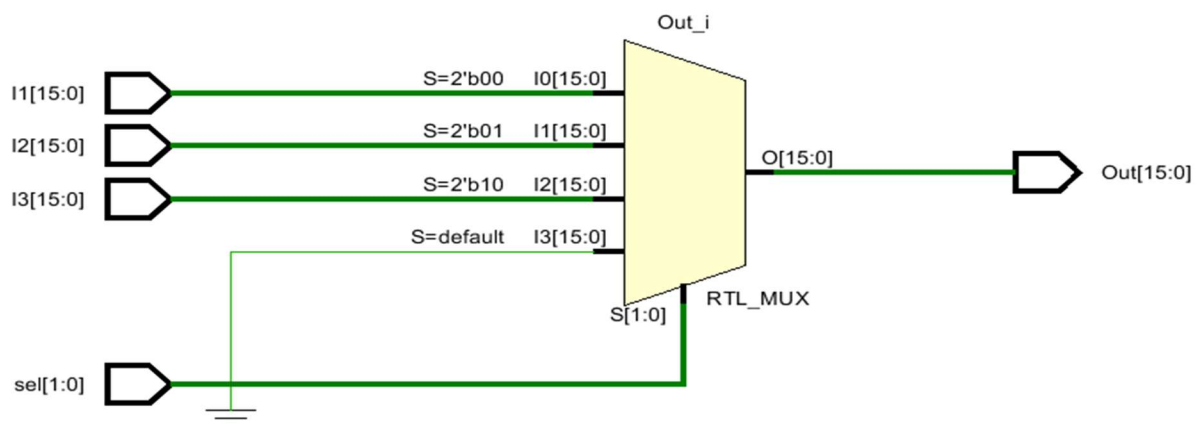
Introduction:

Design Approach:

This project implements a **16-bit Arithmetic Logic Unit (ALU)** using Verilog. The ALU is structured in a modular way to improve readability. The design is divided into **three main functional blocks**:

- **Arithmetic Block** – Implements increment, decrement, addition, addition-with-carry, subtraction, and subtraction-with-borrow. It also generates all arithmetic-related flags: Carry (C), Zero (Z), Negative (N), Overflow (V), Parity (P), and Auxiliary Carry (A).
- **Logic Block** – Performs bitwise operations such as AND, OR, XOR, and NOT. Only logic-relevant flags (Z, N, P) are updated. The unused flags (C, V, A) are assigned “don’t care” values for clarity.
- **Shift/Rotate Block** – Supports logical shifts (SHL, SHR), arithmetic shifts (SAL, SAR), rotates (ROL, ROR), and rotate-through-carry operations (RCL, RCR). This block updates the C, Z, N, and P flags, while V and A are unused.

To select which block’s output becomes the final ALU output, the design uses **two parameterized 3-to-1 multiplexers**: one multiplexer selects the **16-bit result**, and the second selects the **6-bit status flags**.



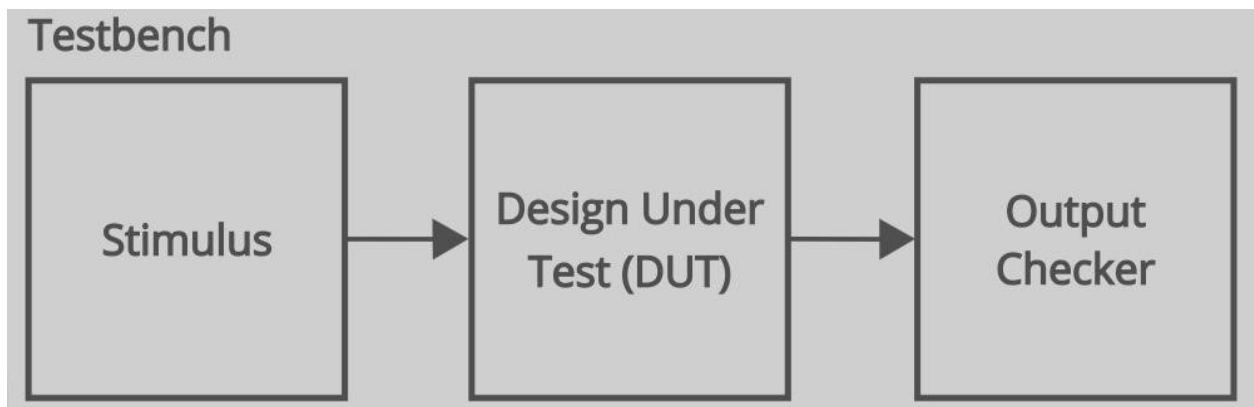
Parameterized mux elaborated design

Verification Approach:

A self-checking Verilog testbench is used to verify the complete functionality of the 16-bit ALU. The testbench applies **directed input vectors** covering all ALU operation categories:

- **Logic operations** (AND, OR, XOR, NOT) are validated using fixed bit patterns to ensure correct bit-wise behavior and accurate updates of the Zero, Negative, and Parity flags.
- **Shift and rotate operations** (SHL, SHR, SAL, SAR, ROL, ROR, RCL, RCR) are tested with boundary bits to confirm correct bit movement, carry extraction, sign extension, and carry-in usage.
- **Arithmetic operations** (INC, DEC, ADD, ADD-with-carry, SUB, SUB-with-borrow) are checked for correct results and proper generation of all related flags, including Carry, Overflow, Auxiliary Carry, Zero, Negative, and Parity.

For every opcode, the testbench manually computes the expected result and status flags. The DUT outputs are then compared against these expected values, and a clear **PASS/FAIL** message is printed. Any mismatch causes the simulation to halt immediately. This verification method guarantees full functional correctness of the ALU, including the Datapath operations, flag logic, and block-selection mechanism via the function code.



Testbench Concept

```

1 module MUX_3_to_1(I1,I2,I3,sel,Out); //parameterized mux for both results(16-bits) and status(6-bits)
2 parameter data_size=16; //default data size
3 input [data_size-1:0] I1,I2,I3;
4 input [1:0] sel;
5 output reg [data_size-1:0] Out;
6 always @(*) begin
7     case(sel)
8         2'b00: Out=I1;
9         2'b01: Out=I2;
10        2'b10: Out=I3;
11        default: Out=0;
12    endcase
13 end
14 endmodule
15 //Arithmetic Block
16 module Arithmetic_block (X,Y,Cin,control_lines,OUT,C,Z,N,V,P,A);
17 input [15:0] X,Y;
18 input Cin;
19 input [2:0] control_lines;
20 output reg [15:0] OUT;
21 output reg C,Z,N,V,P,A;
22 reg [16:0] out_with_carry;
23 reg [15:0] carryout14_bus; //The least significant 14 bits to compute a carry-out (carry-into MSB) to compute The overf
24 reg carryout_of_bit_14; //carry-into the MSB.
25 reg [4:0] summation_first_4bits_with_carry; //The least significant 4 bits used to compute the Auxillary flag(A).
26 reg [4:0] subtraction_first_4bits; //The least significant 4 bits used to compute the Auxillary flag(A).
27 //Arithmetic block
28 always @(*) begin
29     case(control_lines)
30         3'b001: begin //Increment operation
31             OUT=X+1;
32             C=&X;
33             Z=~|OUT;
34             N=OUT[15];
35             V=OUT[15]^X[15];
36             P=~^OUT;
37             A=&X[3:0];
38         end
39         3'b011: begin //Decrement operation
40             OUT=X-1;
41             C=~|X;
42             Z=~|OUT;
43             N=OUT[15];
44             V=OUT[15]^X[15];
45             P=~^OUT;
46             A=~|X[3:0];
47         end
48         3'b100: begin //ADD operation
49             out_with_carry={1'b0,X[15:0]} + {1'b0,Y[15:0]};
50             OUT=out_with_carry[15:0];
51             C=out_with_carry[16];
52             carryout14_bus={1'b0,X[14:0]}+ {1'b0,Y[14:0]};
53             carryout_of_bit_14=carryout14_bus[15];
54             Z=~|OUT;
55             N=OUT[15];
56             V=carryout_of_bit_14 ^ C;
57             P=~^OUT;
58             summation_first_4bits_with_carry={1'b0,X[3:0]} +{1'b0,Y[3:0]};
59             A=summation_first_4bits_with_carry[4];
60         end
61         3'b101: begin //ADD With Carry operation
62             out_with_carry={1'b0,X[15:0]} + {1'b0,Y[15:0]} + Cin;
63             OUT=out_with_carry[15:0];
64             C=out_with_carry[16];
65             Z=~|OUT;
66             N=OUT[15];
67             carryout14_bus={1'b0,X[14:0]}+ {1'b0,Y[14:0]};
68             carryout_of_bit_14=carryout14_bus[15];
69             V=carryout_of_bit_14 ^ C;
70             P=~^OUT;
71             summation_first_4bits_with_carry={1'b0,X[3:0]} +{1'b0,Y[3:0]}+ Cin;
72             A=summation_first_4bits_with_carry[4];
73         end
74         3'b110: begin //SUB
75             out_with_carry={1'b0,X[15:0]}-{1'b0,Y[15:0]};
76             OUT=out_with_carry[15:0];
77             C=out_with_carry[16];
78             carryout14_bus={1'b0,X[14:0]} - {1'b0,Y[14:0]};
79             carryout_of_bit_14=carryout14_bus[15];
80             Z=~|OUT;

```

```

81     N=OUT[15];
82     V=carryout_of_bit_14 ^ C;
83     P=~^OUT;
84     subtraction_first_4bits={1'b0,X[3:0]}-{1'b0,Y[3:0]};
85     A=subtraction_first_4bits[4];
86 end
87 3'b111: begin          //SUB_BORROW
88     out_with_carry={1'b0,X[15:0]}-{1'b0,Y[15:0]} - Cin;
89     OUT=out_with_carry[15:0];
90     C=out_with_carry[16];
91     carryout14_bus={1'b0,X[14:0]} - {1'b0,Y[14:0]} - Cin;
92     carryout_of_bit_14=carryout14_bus[15];
93     Z=~|OUT;
94     N=OUT[15];
95     V=carryout_of_bit_14 ^ C;
96     P=~^OUT;
97     subtraction_first_4bits={1'b0,X[3:0]}-{1'b0,Y[3:0]} - Cin;
98     A=subtraction_first_4bits[4];
99 end
100 endcase
101 end
102 endmodule
103 //logic block
104 module logic_block(X,Y,control_lines,OUT,C,Z,N,V,P,A);
105 input  [15:0] X,Y;
106 input  [2:0] control_lines;
107 output reg [15:0] OUT;
108 output reg C,Z,N,V,P,A;
109 always @(*) begin
110     C=1'bx;          //Unused flag
111     V=1'bx;          //Unused flag
112     A=1'bx;          //Unused flag
113     case(control_lines)
114     3'b000: OUT=X&Y;      //AND
115     3'b001: OUT=X|Y;      //OR
116     3'b010: OUT=X^Y;      //XOR
117     3'b011: OUT=~X;       //NOT
118     default: OUT=16'b0;
119     endcase
120     Z=~|OUT;
121     N=OUT[15];
122     P=~^OUT;
123 end
124 endmodule
125 //shift block
126 module shift_block(X,control_lines,Cin,OUT,C,Z,N,V,P,A);
127 input  [15:0] X;
128 input [2:0] control_lines;
129 input Cin;
130 output reg [15:0] OUT;
131 output reg C,Z,N,V,P,A;
132 reg MSB;
133 reg LSB;
134 always @(*) begin
135     case(control_lines)
136     3'b000: begin          //SHL
137         C=X[15];
138         OUT={X[14:0],1'b0};
139     end
140     3'b001: begin          //SHR
141         C=X[0];
142         OUT={1'b0,X[15:1]};
143     end
144     3'b010: begin          //SAL
145         C=X[15];
146         OUT={X[14:0],1'b0};
147     end
148     3'b011: begin          //SAR
149         C=X[0];
150         OUT={X[15],X[15:1]};
151     end
152     3'b100: begin          //ROL
153         C=X[15];
154         OUT={X[14:0],X[15]};
155     end
156     3'b101: begin          //ROR
157         C=X[0];
158         OUT={X[0],X[15:1]};
159     end
160     3'b110: begin          //RCL
161         MSB=X[15];

```

```

162     OUT={X[14:0],Cin};
163     C= MSB;
164     end
165     3'b111: begin                //RCR
166         LSB=X[0];
167         OUT={Cin,X[15:1]};
168         C=LSB;
169     end
170     default: begin
171         C=0;
172         OUT=16'b0;
173     end
174     endcase
175     Z=~|OUT;
176     N=OUT[15];
177     P=~^OUT;
178     V=1'bx;                    //Unused flag
179     A=1'bx;                    //Unused flag
180 end
181 endmodule
182 module ALU (A,B,F,Cin,Result,Status);
183 //INPUTS
184 input [15:0] A,B;
185 input [4:0] F;
186 input Cin;
187 //OUTPUTS
188 output [15:0] Result;
189 output [5:0] Status;
190 //INTERNAL SIGNALS
191 wire [15:0] arithmetic_out;
192 wire [15:0] logic_out;
193 wire [15:0] shift_out;
194 wire [5:0] arithmetic_status;
195 wire [5:0] logic_status;
196 wire [5:0] shift_status;
197 //ALU BLOCKS
198 Arithmetic_block Arithmetic
199 (.X(A),.Y(B),.Cin(Cin),.control_lines(F[2:0]),.OUT(arithmetic_out),.C(arithmetic_status[5]),.Z(arithmetic_status[4]),.N(arithmetic_status[3])
200 logic_block Logic (.X(A),.Y(B),.control_lines(F[2:0]),.OUT(logic_out),.C(logic_status[5]),.Z(logic_status[4]),.N(logic_status[3]),.V(logic_st
201 shift_block Shift (.X(A),.control_lines(F[2:0]),.Cin(Cin),.OUT(shift_out),.C(shift_status[5]),.Z(shift_status[4]),.N(shift_status[3]),.V(shif
202 MUX_3_to_1 mux_operation (.I1(arithmetic_out),.I2(logic_out),.I3(shift_out),.sel(F[4:3]),.Out(Result));
203 MUX_3_to_1 #6 mux_status (.I1(arithmetic_status),.I2(logic_status),.I3(shift_status),.sel(F[4:3]),.Out(Status));
204 endmodule

```

```

1  module ALU_tb();
2  reg [15:0] A_tb,B_tb;
3  reg Cin_tb;
4  reg [4:0] F_tb;
5  reg [15:0] result_expected;
6  reg [5:0] status_expected;
7  wire [15:0] result_dut;
8  wire [5:0] status_dut;
9
10 ALU DUT (A_tb,B_tb,F_tb,Cin_tb,result_dut,status_dut);
11 initial begin
12     //Testing Arithmetic block
13     //Testing Increment operation
14     A_tb=16'b0001_1110_0001_0011;
15     F_tb=5'b00001;
16     #10;
17     result_expected=16'b0001_1110_0001_0100;
18     status_expected=6'b000010;
19     if((result_expected==result_dut) && (status_expected==status_dut)) begin
20         $display("CORRECT OUTPUT,INCREMENT operation is working well");
21     end
22     else begin
23         $display("ERROR-INCORRECT OUTPUT,INCREMENT operation failed.");
24         $stop;
25     end
26     //Testing Decrement operation
27     A_tb=16'b1101_0110_0101_0011;
28     F_tb=5'b00011;
29     #10;
30     result_expected=16'b1101_0110_0101_0010;
31     status_expected=6'b001010;
32     if((result_expected==result_dut) && (status_expected==status_dut)) begin
33         $display("CORRECT OUTPUT,DECREMENT operation is working well");
34     end
35     else begin
36         $display("ERROR-INCORRECT OUTPUT,DECREMENT operation failed.");
37         $stop;
38     end
39     //Testing ADD operation
40     A_tb=16'b1111_1111_1111_1100;
41     B_tb=16'b1111_1111_1111_0111;
42     F_tb=5'b00100;
43     #10;
44     result_expected=16'b1111_1111_1111_0011;
45     status_expected=6'b101011;
46     if((result_expected==result_dut) && (status_expected==status_dut)) begin
47         $display("CORRECT OUTPUT,ADD operation is working well");
48     end
49     else begin
50         $display("ERROR-INCORRECT OUTPUT,ADD operation failed.");
51         $stop;
52     end
53     //Testing ADD WITH CARRYIN operation

```

```

54 A_tb=16'b1001_1111_0011_0101;
55 B_tb=16'b0110_0001_1111_0111;
56 F_tb=5'b00101;
57 Cin_tb=1;
58 #10;
59 result_expected=16'b0000_0001_0010_1101;
60 status_expected=6'b100000;
61 if((result_expected==result_dut) && (status_expected==status_dut)) begin
62     $display("CORRECT OUTPUT,ADD_CARRY operation is working well");
63 end
64 else begin
65     $display("ERROR-INCORRECT OUTPUT,ADD_CARRY operation failed.");
66     $stop;
67 end
68 //Testing SUB operation
69 A_tb=16'b0001_1100_1101_0010;
70 B_tb=16'b1011_1001_0000_1110;
71 F_tb=5'b00110;
72 #10;
73 result_expected=16'b0110_0011_1100_0100;
74 status_expected=6'b100001;
75 if((result_expected==result_dut) && (status_expected==status_dut)) begin
76     $display("CORRECT OUTPUT,SUB operation is working well");
77 end
78 else begin
79     $display("ERROR-INCORRECT OUTPUT,SUB operation failed.");
80     $stop;
81 end
82 //Testing SUB WITH BORROW operation
83 A_tb=16'b0001_1100_0001_0011;
84 B_tb=16'b1011_1001_1111_0010;
85 F_tb=5'b00111;
86 Cin_tb=1;
87 #10;
88 result_expected=16'b0110_0010_0010_0000;
89 status_expected=6'b100010;
90 if((result_expected==result_dut) && (status_expected==status_dut)) begin
91     $display("CORRECT OUTPUT,SUB_BORROW operation is working well");
92 end
93 else begin
94     $display("ERROR-INCORRECT OUTPUT,SUB_BORROW operation failed.");
95     $stop;
96 end
97 //Testing logic block
98 //Testing AND operation
99 A_tb=16'b0001_1100_1101_0010;
100 B_tb=16'b1011_1001_0000_1110;
101 F_tb=5'b01000;
102 #10;
103 result_expected=16'b0001_1000_0000_0010;
104 status_expected[4]=0; status_expected[3]=0; status_expected[1]=0;
105 if((result_dut==result_expected) && (status_expected[4]==status_dut[4]) &&
(status_expected[3]==status_dut[3]) && (status_expected[1]==status_dut[1])) begin
106     $display("CORRECT OUTPUT,AND operation is working well.");

```

```

107     end
108     else begin
109         $display("ERROR-INCORRECT OUTPUT,AND operation failed.");
110         $stop;
111     end
112     //Testing OR operation
113     A_tb=16'b1010_1111_0001_1001;
114     B_tb=16'b0101_0101_1111_0010;
115     F_tb=5'b01001;
116     #10;
117     result_expected=16'b1111_1111_1111_1011;
118     status_expected[4]=0; status_expected[3]=1; status_expected[1]=0;
119     if((result_dut==result_expected) && (status_expected[4]==status_dut[4]) &&
120 (status_expected[3]==status_dut[3]) && (status_expected[1]==status_dut[1])) begin
121         $display("CORRECT OUTPUT,OR operation is working well.");
122     end
123     else begin
124         $display("ERROR-INCORRECT OUTPUT,OR operation failed.");
125         $stop;
126     end
127     //Testing XOR operation
128     A_tb=16'b0101_1000_1001_1010;
129     B_tb=16'b1001_1011_1000_1110;
130     F_tb=5'b01010;
131     #10;
132     result_expected=16'b1100_0011_0001_0100;
133     status_expected[4]=0; status_expected[3]=1; status_expected[1]=1;
134     if((result_dut==result_expected) && (status_expected[4]==status_dut[4]) &&
135 (status_expected[3]==status_dut[3]) && (status_expected[1]==status_dut[1])) begin
136         $display("CORRECT OUTPUT,XOR operation is working well.");
137     end
138     else begin
139         $display("ERROR-INCORRECT OUTPUT,XOR operation failed.");
140         $stop;
141     end
142     //Testing NOT operation
143     A_tb=16'b0111_1011_1001_0010;
144     F_tb=5'b01011;
145     #10;
146     result_expected=16'b1000_0100_0110_1101;
147     status_expected[4]=0; status_expected[3]=1; status_expected[1]=0;
148     if((result_dut==result_expected) && (status_expected[4]==status_dut[4]) &&
149 (status_expected[3]==status_dut[3]) && (status_expected[1]==status_dut[1])) begin
150         $display("CORRECT OUTPUT,NOT operation is working well.");
151     end
152     else begin
153         $display("ERROR-INCORRECT OUTPUT,NOT operation failed.");
154         $stop;
155     end
156     //Testing Shift Block
157     //Testing SHL
158     A_tb=16'b0011_0100_1111_0010;
159     F_tb=5'b10000;
160     #10;

```

```

158     result_expected=16'b0110_1001_1110_0100;
159     status_expected[5]=0; status_expected[4]=0; status_expected[3]=0;
status_expected[1]=1;
160     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1])) begin
161         $display("CORRECT OUTPUT,SHL operation is working well");
162     end
163     else begin
164         $display("ERROR-INCORRECT OUTPUT,SHL operation failed.");
165         $stop;
166     end
167     //Testing SHR
168     A_tb=16'b1011_1001_1011_0011;
169     F_tb=5'b10001;
170     #10;
171     result_expected=16'b0101_1100_1101_1001;
172     status_expected[5]=1; status_expected[4]=0; status_expected[3]=0;
status_expected[1]=0;
173     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1]))begin
174         $display("CORRECT OUTPUT,SHR operation is working well");
175     end
176     else begin
177         $display("ERROR-INCORRECT OUTPUT,SHR operation failed.");
178         $stop;
179     end
180     //Testing SAL
181     A_tb=16'b1010_1101_0011_0000;
182     F_tb=5'b10010;
183     #10;
184     result_expected=16'b0101_1010_0110_0000;
185     status_expected[5]=1; status_expected[4]=0; status_expected[3]=0;
status_expected[1]=1;
186     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1])) begin
187         $display("CORRECT OUTPUT,SAL operation is working well");
188     end
189     else begin
190         $display("ERROR-INCORRECT OUTPUT,SAL operation failed.");
191         $stop;
192     end
193     //Testing SAR
194     A_tb=16'b1010_0001_1011_0100;
195     F_tb=5'b10011;
196     #10;
197     result_expected=16'b1101_0000_1101_1010;
198     status_expected[5]=0; status_expected[4]=0; status_expected[3]=1;
status_expected[1]=1;
199     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1])) begin
200         $display("CORRECT OUTPUT,SAR operation is working well");

```

```

201     end
202     else begin
203         $display("ERROR-INCORRECT OUTPUT,SAR operation failed.");
204         $stop;
205     end
206     //Testing ROL
207     A_tb=16'b0010_1111_1010_0101;
208     F_tb=5'b10100;
209     #10;
210     result_expected=16'b0101_1111_0100_1010;
211     status_expected[5]=0; status_expected[4]=0; status_expected[3]=0;
status_expected[1]=0;
212     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1])) begin
213         $display("CORRECT OUTPUT,ROL operation is working well");
214     end
215     else begin
216         $display("ERROR-INCORRECT OUTPUT,ROL operation failed.");
217         $stop;
218     end
219     //Testing ROR
220     A_tb=16'b1011_1100_0011_1000;
221     F_tb=5'b10101;
222     #10;
223     result_expected=16'b0101_1110_0001_1100;
224     status_expected[5]=0; status_expected[4]=0; status_expected[3]=0;
status_expected[1]=1;
225     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1])) begin
226         $display("CORRECT OUTPUT,ROR operation is working well");
227     end
228     else begin
229         $display("ERROR-INCORRECT OUTPUT,ROR operation failed.");
230         $stop;
231     end
232     //Testing RCL
233     A_tb=16'b1000_0000_0000_0000;
234     F_tb=5'b10110;
235     Cin_tb=0;
236     #10;
237     result_expected=16'b0000_0000_0000_0000;
238     status_expected[5]=1; status_expected[4]=1; status_expected[3]=0;
status_expected[1]=1;
239     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1])) begin
240         $display("CORRECT OUTPUT,RCL operation is working well");
241     end
242     else begin
243         $display("ERROR-INCORRECT OUTPUT,RCL operation failed.");
244         $stop;
245     end
246     //Testing RCR

```

```

247     A_tb=16'b1000_0110_0000_0001;
248     F_tb=5'b10111;
249     Cin_tb=1;
250     #10;
251     result_expected=16'b1100_0011_0000_0000;
252     status_expected[5]=1; status_expected[4]=0; status_expected[3]=1;
status_expected[1]=1;
253     if((result_dut==result_expected) && (status_expected[5]==status_dut[5]) &&
(status_expected[4]==status_dut[4]) && (status_expected[3]==status_dut[3]) &&
(status_expected[1]==status_dut[1])) begin
254         $display("CORRECT OUTPUT,RCR operation is working well");
255     end
256     else begin
257         $display("ERROR-INCORRECT OUTPUT,RCR operation failed.");
258         $stop;
259     end
260     $display("Your ALU is ready!!!");
261     $stop;
262 end
263 endmodule

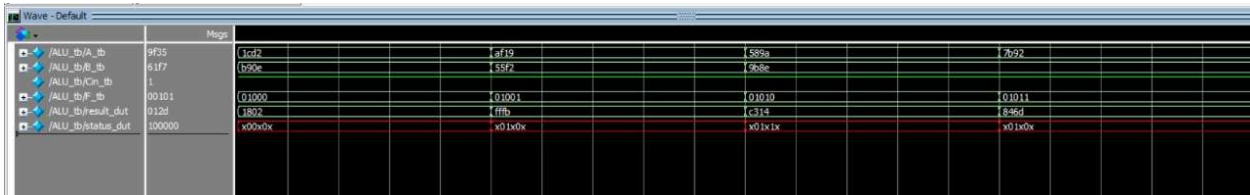
```

Waveform:

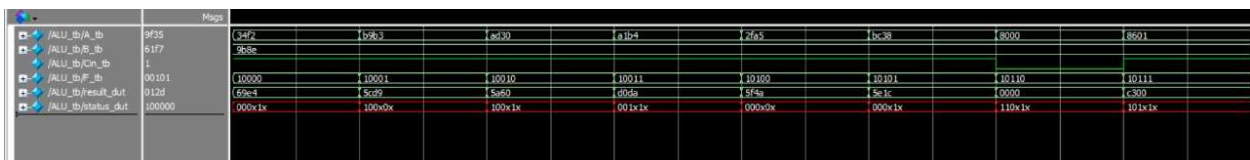
Arithmetic block waveform:



Logic block waveform:



Shift block waveform:

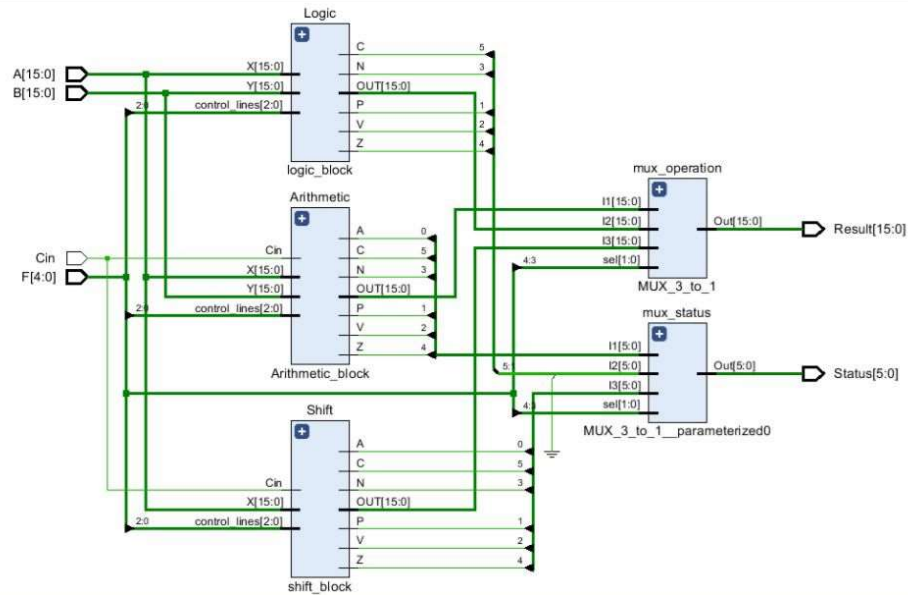


Transcript:

```
# CORRECT OUTPUT,INCREMENT operation is working well
# CORRECT OUTPUT,DECREMENT operation is working well
# CORRECT OUTPUT,ADD operation is working well
# CORRECT OUTPUT,ADD_CARRY operation is working well
# CORRECT OUTPUT,SUB operation is working well
# CORRECT OUTPUT,SUB_BORROW operation is working well
# CORRECT OUTPUT,AND operation is working well.
# CORRECT OUTPUT,OR operation is working well.
# CORRECT OUTPUT,XOR operation is working well.
# CORRECT OUTPUT,NOT operation is working well.
# CORRECT OUTPUT,SHL operation is working well
# CORRECT OUTPUT,SHR operation is working well
# CORRECT OUTPUT,SAL operation is working well
# CORRECT OUTPUT,SAR operation is working well
# CORRECT OUTPUT,ROL operation is working well
# CORRECT OUTPUT,ROR operation is working well
# CORRECT OUTPUT,RCL operation is working well
# CORRECT OUTPUT,RCR operation is working well
# Your ALU is ready!!!
```

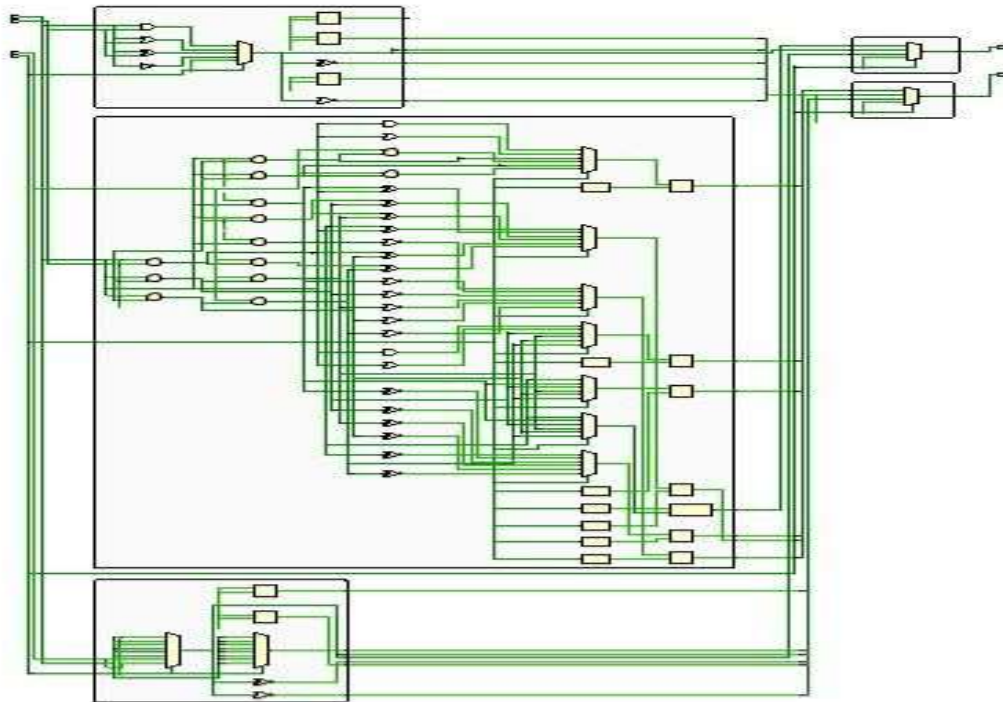
Transcript taken from Questa-Sim

Elaborated Design:



Elaborated design schematic taken from vivado

Detailed Elaborated Design:



Detailed Elaborated Design taken from Vivado

Conclusion:

A modular 16-bit ALU was successfully designed and verified using Verilog. The architecture was divided into Arithmetic, Logic, and Shift blocks, with parameterized multiplexers combining their outputs into a flexible top-level unit. A comprehensive self-checking testbench validated all operations and status flags, confirming correct functionality across the entire design. The results demonstrate a reliable, expandable ALU suitable for integration into larger digital systems.